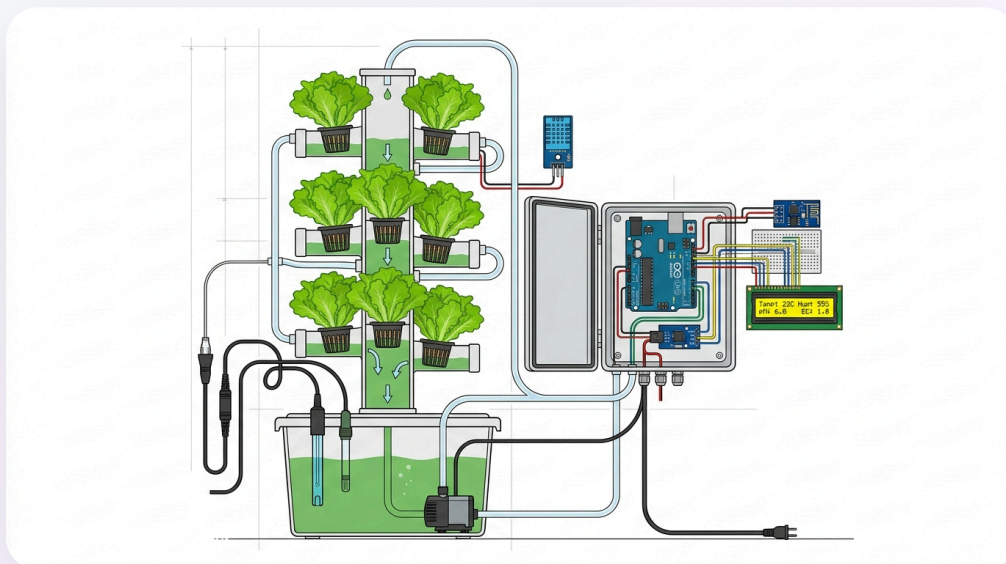


TECHNICAL DOCUMENTATION

Smart Indoor Garden

Data Collection & AI Training Guide



Project: Predictive Hydroponic Optimization

Institution: YSK 2026 — Kisii School

Platform: Arduino Uno + ESP-01 (ESP8266) Edge AI

Date: August 2026

1. System Overview

The Smart Indoor Garden system uses a **two-microcontroller architecture** to monitor hydroponic conditions, collect calibration data, and deploy machine learning models for automated nutrient dosing optimization.

Arduino Uno — Sensor Node

Acts as a dedicated peripheral for data acquisition and actuator control:

- **DHT11 Sensor** (Pin 5) — Temperature & Humidity
- **pH Sensor** (Simulated digitally via mathematical model)
- **EC/Nutrient Sensor** (Analog A1) — Nutrient concentration
- **Submersible Pump** (Pin 9) — Nutrient dosing actuator
- **Piezo Buzzer** (Pin 11) — Audio alerts
- **16×2 LCD Display** (I²C 0x27) — Local readout

ESP-01 — Master Controller

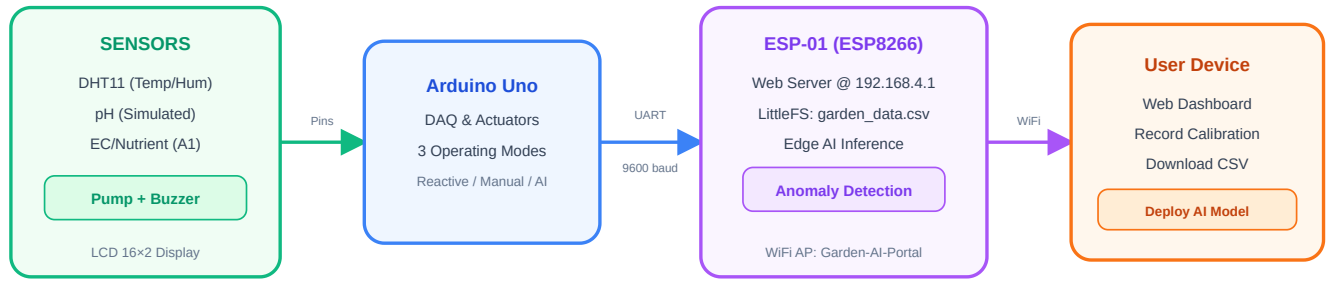
Serves as the brain and connectivity hub:

- **WiFi Access Point** — `Garden-AI-Portal`
- **Web Dashboard** — Real-time monitoring at `192.168.4.1`
- **LittleFS Storage** — Calibration CSV & model params
- **Edge AI Engine** — Linear Regression / MLP inference
- **Anomaly Detection** — Sensor drift & pump failure monitoring
- **UART Bridge** — Serial commands to/from Arduino

COMMUNICATION PROTOCOL

The Arduino sends telemetry packets to the ESP-01 via UART serial at 9600 baud every **500ms** in the format: `DATA:temp,hum,ph,nc,pumpState` . The ESP-01 sends actuator commands back (e.g., `PUMP:ON` , `DOSING:5.0` , `MODE:AI`).

System Architecture



2. Data Collection Process

Data collection is performed through the **Garden-AI web dashboard**, where the user manually logs calibration data points that pair live sensor readings with an appropriate dosing duration. This builds the training dataset for the Edge AI model.



Figure 1: Garden-AI Portal Web Dashboard — accessed at 192.168.4.1

1

Connect to the Garden Portal

On your phone or laptop, connect to the WiFi network:

- **SSID:** Garden-AI-Portal
- **Password:** garden1234

Open a web browser and navigate to **192.168.4.1** to access the real-time dashboard.

2

Switch to Manual Mode

In the **System Control Center** panel, click the **"Manual"** mode button. This enables direct control of the pump and the dosing duration slider. The system has three operating modes:

- **Reactive** — Automatic dosing based on threshold rules ($NC \geq 507$ or $NC < 50$)

- **Manual** — Full user control over pump and dosing for calibration
- **AI-Edge** — Automated dosing predictions from the deployed ML model

3

Set the Dosing Duration

Use the "**Target Dosing Duration**" slider (range: 0.5s — 30.0s, in 0.5s steps) to set how long the pump *should* run given the current environmental conditions. This is the **target label** for the machine learning model — the human-expert judgment of the correct dosing time.

4

Record the Calibration Point

Click the "**Record Calibration**" button. The system captures the current live sensor readings and pairs them with your chosen dosing duration, saving a row to the CSV file on the ESP-01's flash filesystem (LittleFS).

IMPORTANT

Sensor data is **read automatically** from the Arduino co-processor in real time — you do *not* manually type sensor values. The only user input is the **dosing duration** slider, which represents your expert judgment for the given conditions.

3. Calibration Dataset Structure

Each time the user presses **"Record Calibration"**, a new row is appended to the file `/garden_data.csv` on the ESP-01's LittleFS filesystem. The CSV uses 5 columns:

Column	Data Type	Source	Range	Description
<code>temp</code>	Float (1 d.p.)	DHT11 sensor	0.0 — 50.0 °C	Ambient air temperature near the grow zone
<code>hum</code>	Integer	DHT11 sensor	0 — 100 %	Relative humidity percentage
<code>ph</code>	Float (2 d.p.)	Simulated (digital twin)	0.00 — 14.00	Nutrient solution pH level
<code>nc</code>	Integer	Analog pin A1	0 — 1023	Nutrient concentration (EC proxy) — raw ADC value
<code>dosing_duration</code>	Float (1 d.p.)	User input (slider)	0.5 — 30.0 s	Target pump run time set by the user

Example CSV Output

```
// File: /garden_data.csv (stored on ESP-01 LittleFS)
temp,hum,ph,nc,dosing_duration
25.3,68,6.42,312,5.0
26.1,72,6.85,189,8.5
24.7,65,7.10,520,2.0
27.0,58,5.90,410,3.5
23.5,80,6.55,275,6.0
```

Dataset Management

STORAGE SAFEGUARDS

File size cap: 50 KB maximum — the system rejects new entries once this limit is reached to protect the ESP-01's limited flash memory.

USER ACTIONS

Download CSV: Click the "Download" button to export the file via the `/download_csv` endpoint.

Heap check: Logging is blocked if free heap drops below 6,144 bytes.

Disk check: Logging is blocked if free filesystem space falls below 15,360 bytes.

Clear All Data: Click "Clear" to permanently delete all calibration data and start fresh.

Minimum required: At least **5 calibration points** are needed before the AI can be trained.

pH Simulation Model

Since no physical pH probe is connected, the Arduino mathematically simulates realistic pH behavior using a **digital twin model**:

- **Upward drift:** When the pump is OFF, pH increases by +0.02 units every 2 seconds (simulating plant nutrient uptake), capped at pH 8.2.
- **Dosing correction:** When the pump is ON, pH moves toward 6.0 at a rate of ± 0.15 units per step (simulating acid/base nutrient dosing).
- **Measurement noise:** Random micro-fluctuations of ± 0.02 are added to simulate real sensor jitter.

4. Data-to-Deployment Pipeline

Once calibration data is collected, it flows through a machine learning pipeline to produce an optimized Edge AI model that runs directly on the ESP-01 microcontroller.

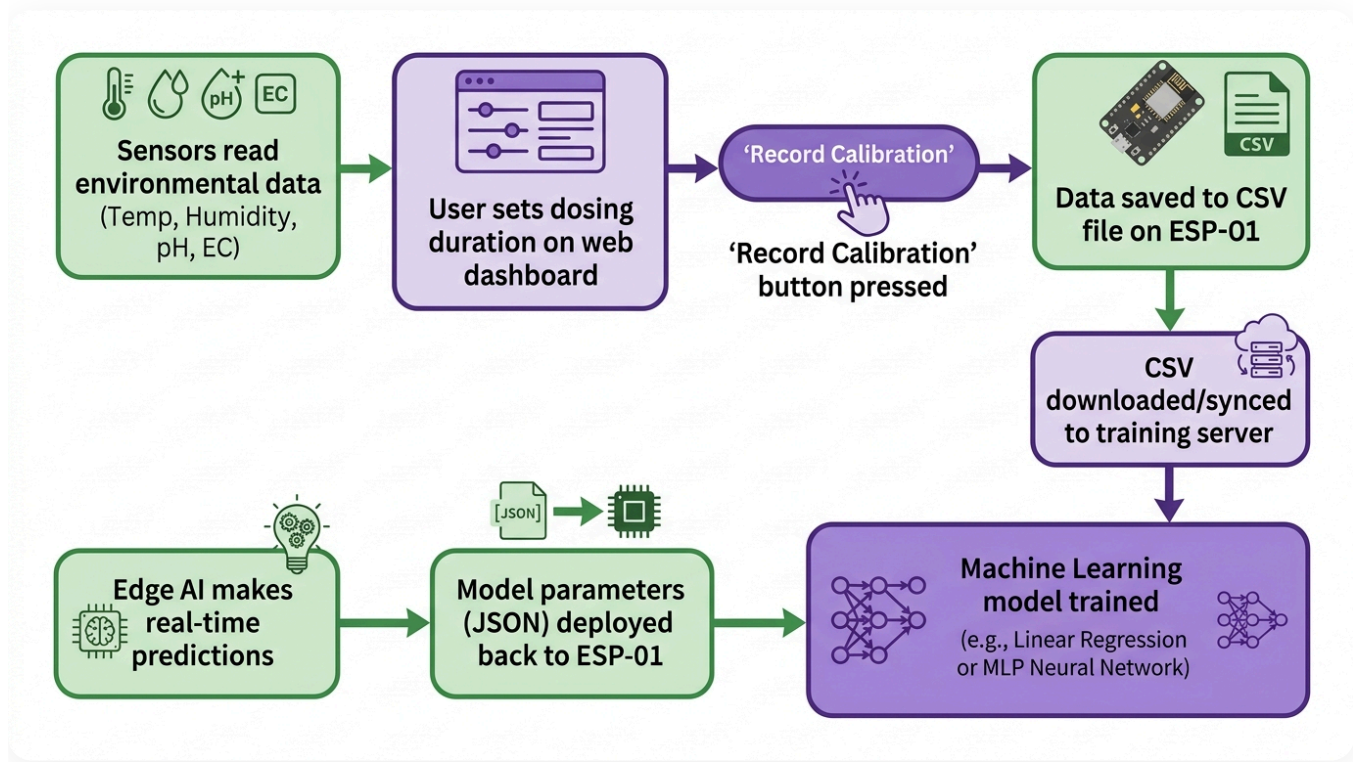


Figure 2: End-to-end data collection and AI deployment pipeline

Method A: Manual Training via Garden-AI Studio

1

Export the Dataset

Download `garden_data.csv` from the dashboard's "Download" button (or visit `192.168.4.1/download_csv`).

2

Upload to the Training Server

Open the **Garden-AI Training Studio** web application (Python Flask server running on `app.py`). Upload the CSV file and select one of two regression algorithms:

- **Multivariate Linear Regression** — Fast, interpretable, fewer parameters

- **Multi-layer Perceptron (MLP)** — 8-neuron hidden layer, ReLU activation, better for non-linear patterns

3

Evaluate & Download Model

The studio displays R^2 Score and Mean Absolute Error (MAE) along with an actual-vs-predicted comparison table. Download the resulting `model_params.json` file.

4

Deploy to ESP-01

In the Garden-AI Dashboard, drag and drop `model_params.json` into the **"Deploy Parameters"** dropzone (or click to upload). The model weights are saved to LittleFS and immediately loaded for real-time inference.

Method B: Automatic Self-Learning Sync Loop

BACKGROUND AUTO-TRAINING

When the Python training server (`app.py`) runs on a device connected to the Garden-AI-Portal WiFi, a **background thread** automatically:

1. Polls the ESP-01 for new CSV data every **10 seconds** via `http://192.168.4.1/download_csv`.
2. Detects new calibration rows (minimum 5 required).
3. Trains an MLP model and evaluates R^2 score.
4. Auto-deploys the new weights to the ESP-01 **only if R^2 improves** over the previous best.
5. Updates the training status on the dashboard in real time (overlay animation + status pill).

Input Normalization

Both training methods apply **Min-Max scaling** to normalize sensor inputs to the 0.0–1.0 range before training and inference:

```
// Normalization formulas (applied identically on ESP-01 and training server)
n_temp = temp / 50.0           // Temperature: max 50°C
n_hum  = hum  / 100.0          // Humidity: max 100%
n_ph   = ph   / 14.0           // pH scale: max 14.0
n_nc   = nc   / 1023.0         // ADC reading: max 1023
```


5. Best Practices for Data Collection

GOAL

Collect a **diverse, representative** dataset that covers the full range of environmental conditions your garden will encounter. The AI model can only predict well for conditions it has seen in training data.

Recommended Collection Strategy

Condition	What to Vary	Example Scenarios	Suggested Dosing
Low Nutrients	NC < 200	Fresh water, diluted reservoir	8–15 seconds (longer dosing)
Optimal Range	NC 200–500	Normal growing conditions	3–6 seconds (maintenance dose)
High Nutrients	NC > 500	Over-fertilized, salt buildup	0.5–2 seconds (minimal or flush)
High pH (> 7.0)	pH drifting upward	Alkaline conditions, aged reservoir	Increase dosing to acidify
Low pH (< 5.8)	pH dropping	Over-corrected, excess acid	Reduce dosing, allow stabilization
Temperature Extremes	Temp < 18°C or > 30°C	Cold mornings, hot afternoons	Adjust based on plant uptake rate

Data Quality Tips

- Collect at different times of day** — temperature and humidity vary throughout the day, giving the model more diverse training examples.
- Aim for 20+ calibration points** — while 5 is the minimum for training, more data points improve model accuracy and generalization.
- Cover edge cases** — deliberately log points when conditions are poor (very high/low NC, extreme pH) so the model learns corrective dosing.
- Be consistent with your dosing judgment** — for similar conditions, assign similar dosing durations to avoid confusing the model with contradictory labels.

5. **Clear and restart if needed** — if early calibration points were inaccurate, use the "Clear" button and start fresh rather than training on noisy data.

Model Selection Guide

LINEAR REGRESSION

Best for: Small datasets (5–15 points), simple linear relationships.

Parameters: 5 weights (intercept + 4 feature weights).

Pros: Fast, interpretable, stable with few data points.

Cons: Cannot capture non-linear sensor interactions.

MLP NEURAL NETWORK

Best for: Larger datasets (15+ points), complex non-linear patterns.

Parameters: 8-neuron hidden layer (49 total parameters).

Pros: Captures non-linear interactions between pH, EC, temp, humidity.

Cons: Requires more data; can overfit on very small datasets.

Operational Limits

SYSTEM CONSTRAINTS

- **CSV file size:** Maximum 50 KB on ESP-01 LittleFS flash storage.
- **AI inference cooldown:** In AI-Edge mode, dosing predictions run every 15 seconds with a 60-second cooldown after each dose to allow nutrient mixing.
- **Dosing range:** AI predictions are constrained to 0.0 — 30.0 seconds. Dosing is only triggered when prediction > 0.5s and NC < 500.

6. Web API Endpoint Reference

The ESP-01 serves a RESTful API for all data collection, model deployment, and system control operations. All endpoints are accessible at <http://192.168.4.1>.

Endpoint	Method	Purpose	Parameters
<code>/</code>	GET	Serve the main dashboard HTML page	—
<code>/status</code>	GET	Return live telemetry JSON (polled every 1s)	—
<code>/mode</code>	POST	Switch operating mode	<code>mode</code> = reactive manual ai
<code>/move</code>	POST	Manual pump control or timed dosing	<code>posH</code> (duration), <code>action</code> (ON/OFF)
<code>/log</code>	POST	Record calibration data point	<code>duration</code> (dosing time in seconds)
<code>/download_csv</code>	GET	Download the full calibration CSV file	—
<code>/clear_log</code>	POST	Delete all calibration data	—
<code>/deploy_model</code>	POST	Upload and activate AI model weights	<code>model_file</code> (JSON string)
<code>/download_model</code>	GET	Download current model_params.json	—
<code>/delete_model</code>	POST	Remove deployed model, revert to Reactive	—
<code>/set_training</code>	POST	Update training status indicator	<code>training</code> (0/1), <code>status</code> (text)

Example: Recording a Data Point Programmatically

```
# Python example: log a calibration point with 7.5s dosing
import requests

response = requests.post(
    "http://192.168.4.1/log",
    params={"duration": "7.5"}
)
print(response.text)
# Output: "Garden data point recorded successfully!"
```

Example: Fetching Live Telemetry

```
# Python example: get current sensor readings
import requests

data = requests.get("http://192.168.4.1/status").json()
print(f"Temp: {data['temp']}°C  pH: {data['ph']}  NC: {data['nc']}")
print(f"Mode: {data['mode']}  Pump: {'ON' if data['pumpState'] else 'OFF'}")
print(f"AI Loaded: {data['ai_loaded']}  Anomaly: {data['anomaly_reason']}")
```